

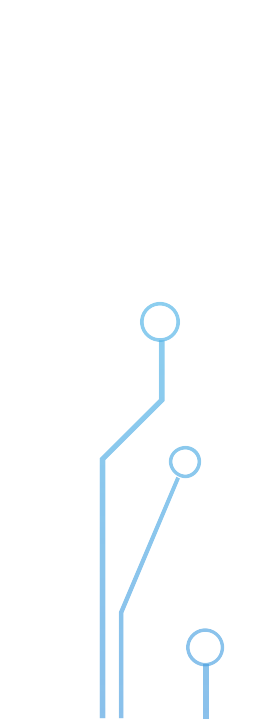
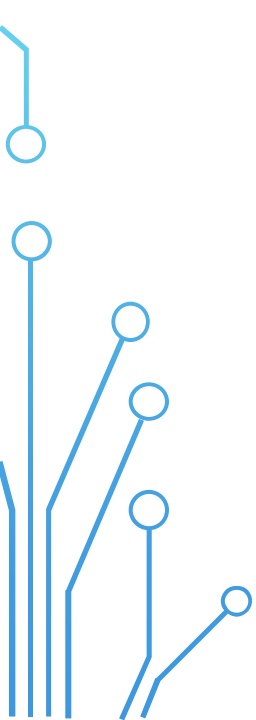
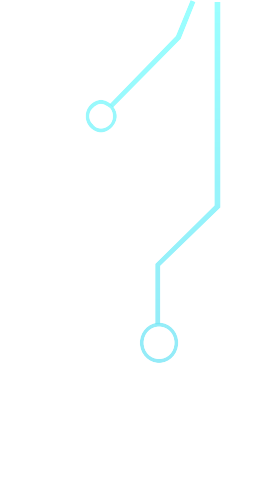
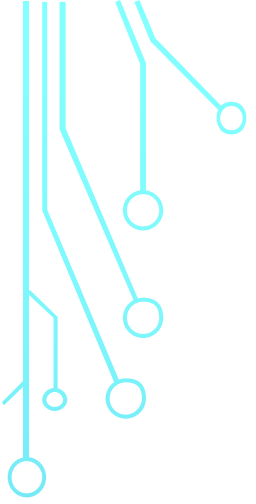
A decorative graphic on the left side of the slide, consisting of a network of thin, gold-colored lines and small circles, resembling a circuit board or a stylized tree structure.

Unix Basics

COMP211 Connor McMahon

Reminders

- HW1 is due today.
- If you have not already signed up for a checkoff slot, do that now!
- If you haven't gotten your setup from HW0 checked, do that today!
- You will complete the checkoff inside your container.



\$PATH

Paths

- We know what an **absolute path** and a **relative path** are.
- An **absolute path** starts from the root. Example:
 - `/home/alice/projects/hello`
- A **relative path** starts from wherever we currently are. Examples:
 - `./hello`
 - `../hello`
 - `projects/hello`

Paths

- Now suppose we've written and compiled a program called `hello`:

```
$ gcc hello.c -o hello
```

- We can run it with:

```
$ ./hello
```

- You might wonder:
 - Why do I need the `./`?

Paths

- Additionally, why do I type:

```
$ ls
```

```
$ pwd
```

```
$ wc
```

- And not

```
$ ./ls
```

```
$ ./pwd
```

```
$ ./wc
```

- That's where `$PATH` comes in.

What does `./hello` actually mean?

- Remember that `.` means the current directory.

- So:

```
$ ./hello
```

- is simply a relative path.
- You're telling the shell:
 - Go into the current directory (`.`), find the file called `hello`, and execute it.
- It's conceptually the same as giving its absolute path:

```
$ /home/alice/projects/hello
```

- Both commands tell the shell **exactly where the executable is located.**

What does ./hello actually mean?

- So what happens if we leave off the path?

```
$ hello
```

- Now we haven't told the shell where `hello` is.
- **When a command contains no /, the shell searches for it.**
- Where does it search?

What does ./hello actually mean?

```
$ ./hello
```

- contains a /
- meaning "Here is a path. Run this file."

```
$ hello
```

- contains no /
- meaning "Find a command named hello for me."

- And where does the shell look?
 - \$PATH

\$PATH is the shell's command search path

- Try:

```
$ echo $PATH
```

- You might see something like:

```
/usr/local/bin:/usr/bin:/bin:/home/alice/
```

- That's a list of directories separated by **:**
- It tells your shell:
 - When I type a command without specifying its location, search these directories for it.

\$PATH is the shell's command search path

- So imagine you type:

```
$ wc
```

- Your shell effectively searches:

```
$ /usr/local/bin/wc
```

```
$ /usr/bin/wc
```

<– found it!

- and then executes the one it found.

\$PATH is the shell's command search path

- So that's why:

```
$ ls
```

- works while:

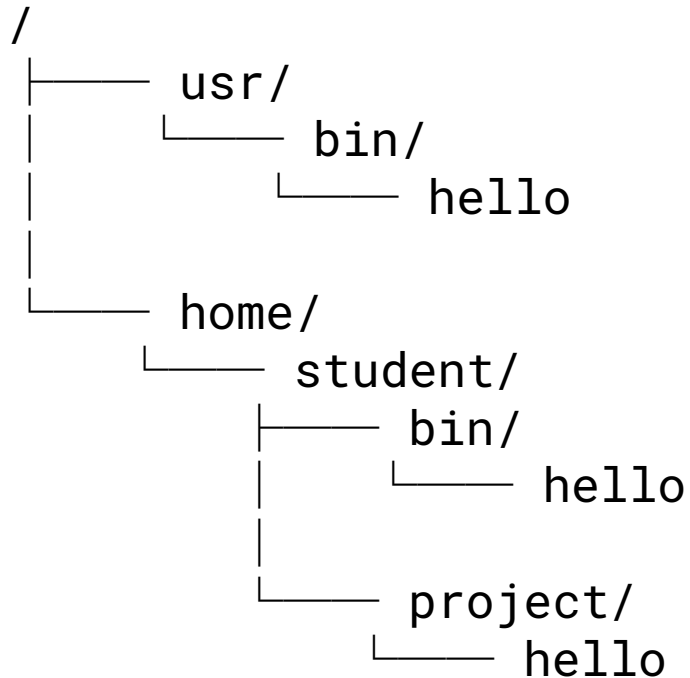
```
$ hello
```

- doesn't.

- Your hello executable is sitting in your current directory, but **your current directory usually isn't in \$PATH.**

Example

- You are currently in: `/home/student/project`
- Your filesystem contains:

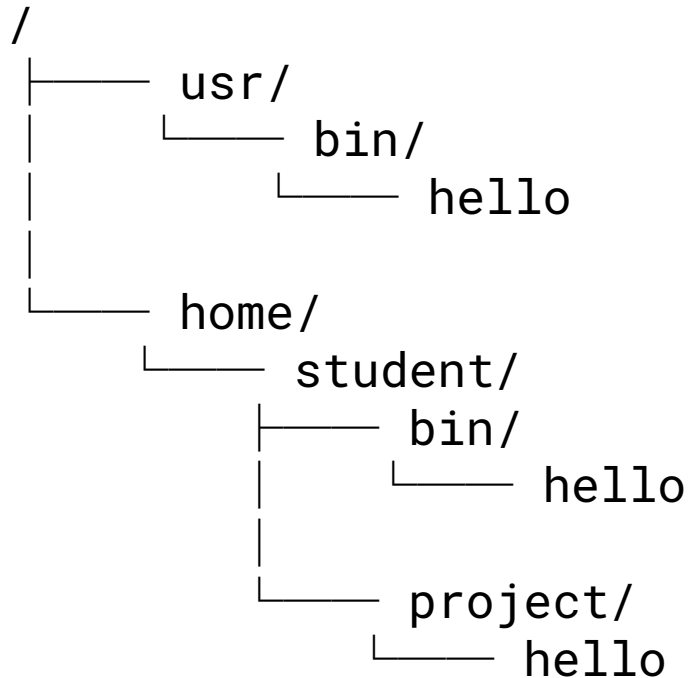


Command	Which hello runs?
<code>./hello</code>	
<code>/usr/bin/hello</code>	
<code>../bin/hello</code>	
<code>hello</code>	

- All three hello files are executable.
- Your `$PATH` is: `/home/student/bin:/usr/bin:/bin`

Example

- You are currently in: `/home/student/project`
- Your filesystem contains:



Command	Which hello runs?
<code>./hello</code>	<code>/home/student/project/hello</code>
<code>/usr/bin/hello</code>	<code>/usr/bin/hello</code>
<code>../bin/hello</code>	<code>/home/student/bin/hello</code>
<code>hello</code>	<code>/home/student/bin/hello</code>

- All three hello files are executable.
- Your `$PATH` is: `/home/student/bin:/usr/bin:/bin`

Active Learning

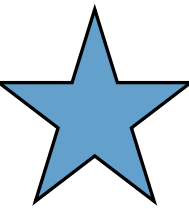
1. Why does `$ ./hello` run the version in `project/`?
2. Why doesn't typing `$ hello` run the version in the current directory?
3. For `$ hello`, why does `/home/student/bin/hello` run instead of `/usr/bin/hello`?
4. Now change `$PATH` to: `/usr/bin:/home/student/bin:/bin`
5. Without moving or modifying any files, what does: `$ hello` run now? Why?
6. How could you modify `$PATH` so that you can type `$ hello` from *any directory* and have `/home/student/project/hello` run, without moving the executable?

Active Learning



1. `.$ ./hello` runs the version in `project/` because `.` means “the current directory.” Since the current directory is `/home/student/project`, `.$ ./hello` refers to: `/home/student/project/hello`
2. Typing `$ hello` does not automatically run the version in the current directory because the shell does not normally search the current directory for commands. Instead, when a command contains no `/`, the shell searches the directories listed in `$PATH`.
3. With: `/home/student/bin:/usr/bin:/bin` the shell searches from left to right. It finds `hello` in: `/home/student/bin` before it ever checks `/usr/bin`, so `/home/student/bin/hello` runs.

Active Learning



4. After changing \$PATH to: `/usr/bin:/home/student/bin:/bin` the search order has changed. `/usr/bin` is now checked before `/home/student/bin`.

5. Now: `hello` runs: `/usr/bin/hello` because the shell searches \$PATH from left to right and finds hello in /usr/bin first.

6. You could add `/home/student/project` to the beginning of \$PATH:

```
export PATH="/home/student/project:$PATH"
```

Then when you type:

```
hello
```

the shell checks `/home/student/project` first and finds:

```
/home/student/project/hello
```

Because that directory is in \$PATH, you can run `hello` from any current working directory.

Why isn't the current directory in \$PATH?

- You might wonder: why doesn't the shell just search the current directory too?
- There's an important security reason.

- Imagine you download some files and enter the directory:

```
$ cd ~/Downloads/suspicious-stuff
```

- Someone has put a malicious executable there called:

```
ls
```

- If the shell always searched the current directory first, then when you innocently typed:

```
$ ls
```

- you could accidentally run:

```
$ ./ls
```

- instead of the real system `ls`.

Why isn't the current directory in \$PATH?

- By requiring:

```
$ ./hello
```

- you're explicitly saying:
- Yes, I purposefully want to execute the hello program located **right here**.

Shell Globbing

Globbering

- **Globbering** is a way of using patterns to match groups of filenames instead of typing them one by one.
- The shell (bash, zsh, etc.) expands these patterns before running the command.

Pattern	Meaning
*	any number of characters
?	exactly one character
[abc]	one character from the set
[a-z]	one character in a range

Globbering

- **Globbering** saves time and reduces mistakes

- For example, instead of typing

```
rm file1.txt file2.txt file3.txt file4.txt
```

- You can type

```
rm *.txt
```

- This command will delete all files in the current working directory that end in **.txt**

Shell Glob Examples

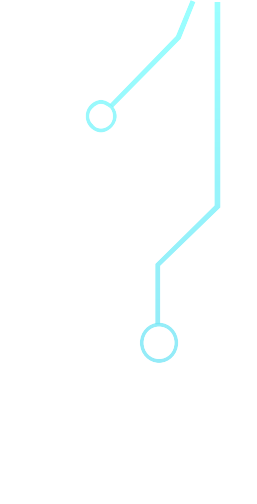
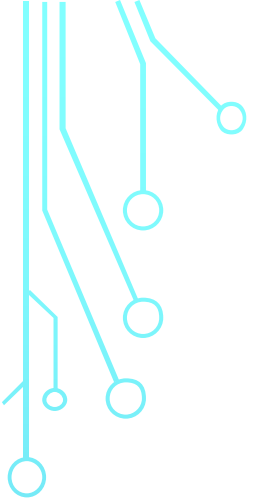
Pattern	Meaning	Expansion Examples
<code>*.txt</code>	Anything ending in <code>.txt</code>	<code>foo.txt</code> <code>a.txt</code>
<code>data_?.?.csv</code>	<code>data_</code> + exactly 2 chars + <code>.csv</code>	<code>data_01.csv</code> <code>data_ab.csv</code> <code>data_XY.csv</code>
<code>[ab]*.md</code>	Starts with a or b, ends with <code>.md</code>	<code>about.md</code> <code>blogpost.md</code> <code>b_notes.md</code>

How does globbing work in the shell?

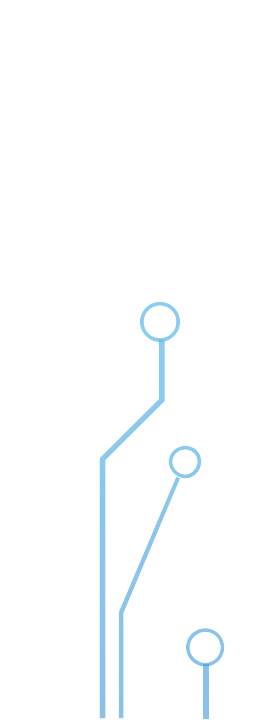
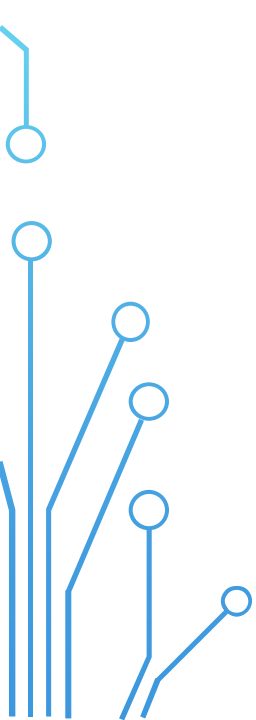
- The shell expands filename patterns before running a command.
- For example, suppose a directory contains:
 - `cat.jpg`
 - `dog.txt`
 - `notes.txt`
 - `photo.png`
- If you type:

```
cat *.txt
```
- The shell sees `*.txt`, searches the current directory, and expands the command to

```
cat dog.txt notes.txt
```
- Then it runs `cat`.
 - Importantly, `cat` itself never sees `*.txt`.



Regular Expressions



Regular Expressions

- A **regular expression (regex)** is a pattern used to search, match, or manipulate text.

Examples of Where Regular Expressions Are Used

- Web Development
 - Form validation
 - URL Routing
 - Input sanitation
 - Scraping
- ML Preprocessing
 - Clean data (e.g. strip HTML tags, normalize whitespace)
 - Extract dates, IDs, and other metadata
- Debugging
 - Find patterns in logs
 - Filter stack traces
 - Search large codebases

Character Class

- A **character class** is a pattern that represents a set of characters and matches exactly one character from that set.

Regex	Meaning
[a b c]	match a or b or c
[a - z]	any lowercase letter
[A - Z]	any uppercase letter
[0 - 9]	any digit

b[ae][dt]

String	Matches?
bat	yes
Bat	no
bad	yes
bed	yes
bea	no
ba	no

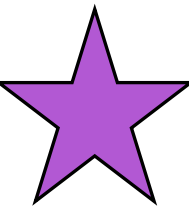
Quantifiers (how many times)

Regex	Meaning
*	0 or more
+	1 or more
?	0 or 1
{n}	exactly n
{n,}	n or more
{n,m}	between n and m

Anchors (position)

Regex	Meaning
^	start of line
\$	end of line

`^ID-[0-9]{3}$`



String	Matches?
ID-123	yes
ID-12	No, only 2 digits
ID-999	yes
abcID-123	No, extra text before
id-123	No, lowercase id
ID-123xyz	No, extra text after

Compare with No Anchors

- Without anchors
 - ID-[0-9]{3}
 - This would match inside:
 - abcID-123xyz
- With anchors, ^ID-[0-9]{3}\$
 - Only matches if the whole string is exactly that pattern.

Escaping

- In regular expressions, the dot (.) means “Match any single character (except newline)”
- Escaping uses an escape character (often `\`) to change how the character or sequence that follows it is interpreted.
- Escaping tells the regex engine: “I want this character itself, not what it normally means.”

Brief Aside: Escaping Is a General Concept

- We have also seen **escaping** with special characters!
- In many programming contexts, certain characters have a special meaning.
- A backslash (\) can be used to change how the character that follows it is interpreted.
- For example, in C:

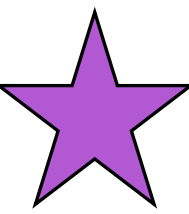
```
printf("Hello\nWorld");
```

 - \n is interpreted as a **newline**, rather than the two characters \ and n.

Regular Expression Example

- Match a string that:
 - Starts with one lowercase letter
 - Then exactly 2 digits
 - Ends with .txt
- Should match:
 - a12.txt
 - z99.txt
- Should NOT match:
 - A12.txt (uppercase)
 - a1.txt (only 1 digit)
 - a123.txt (3 digits)
 - a12.txtt (wrong extension)

Regular Expression Example



- Match a string that:
 - Starts with one lowercase letter
 - Then exactly 2 digits
 - Ends with .txt
- Should match:
 - a12.txt
 - z99.txt
- Should NOT match:
 - A12.txt (uppercase)
 - a1.txt (only 1 digit)
 - a123.txt (3 digits)
 - a12.txtt (wrong extension)

`^[a-z][0-9]{2}\.txt$`

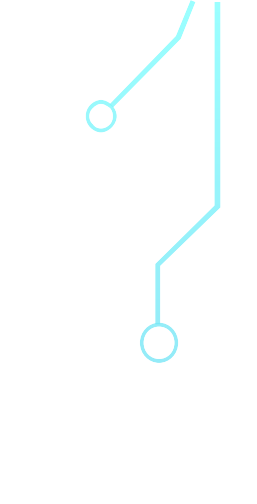
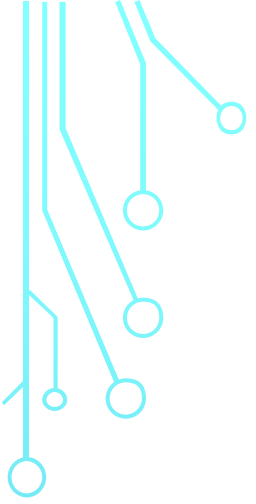
Expansion vs. Matching

- Globbing = expansion
- Regex = matching

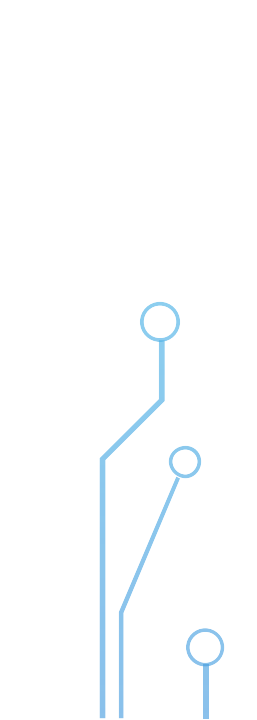
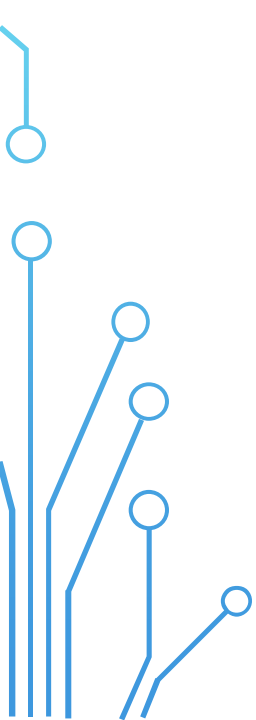
Feature	Globbing	Regex
Operates on	Filenames	Text
Purpose	Select files	Match strings
Done by	Shell	Program
Result	Expanded arguments	True/False matches

Glob vs. Regex Syntax Comparison

Meaning	Glob	Regex
Any sequence	*	.*
Any single char	?	.
Literal dot	.	\.
Character class	[abc]	[abc]
Repetition	(does not exist)	{m,n}, +, *



grep and find



The find Command

- `find` searches the filesystem for files or directories
- It starts at a given directory and searches recursively
- Results are based on conditions such as:
 - File name
 - File type
 - Size, time, permissions (and more)

find -name Format

```
find <where to search> -name <pattern>
```

- Example:

```
find ./ -name dog.c
```

- ./ = start searching from the current directory
- -name = match files by name
- dog.c = exact file name to match

- `find` uses **shell-style glob patterns**, not regular expressions

find Example

```
find ./ -name dog.c
```

- ./ start searching from the current directory
- -name match files by name
- dog.c match exactly files named dog.c
- Output

```
./src/dog.c
```

```
./tests/dog.c
```

```
./dog.c
```

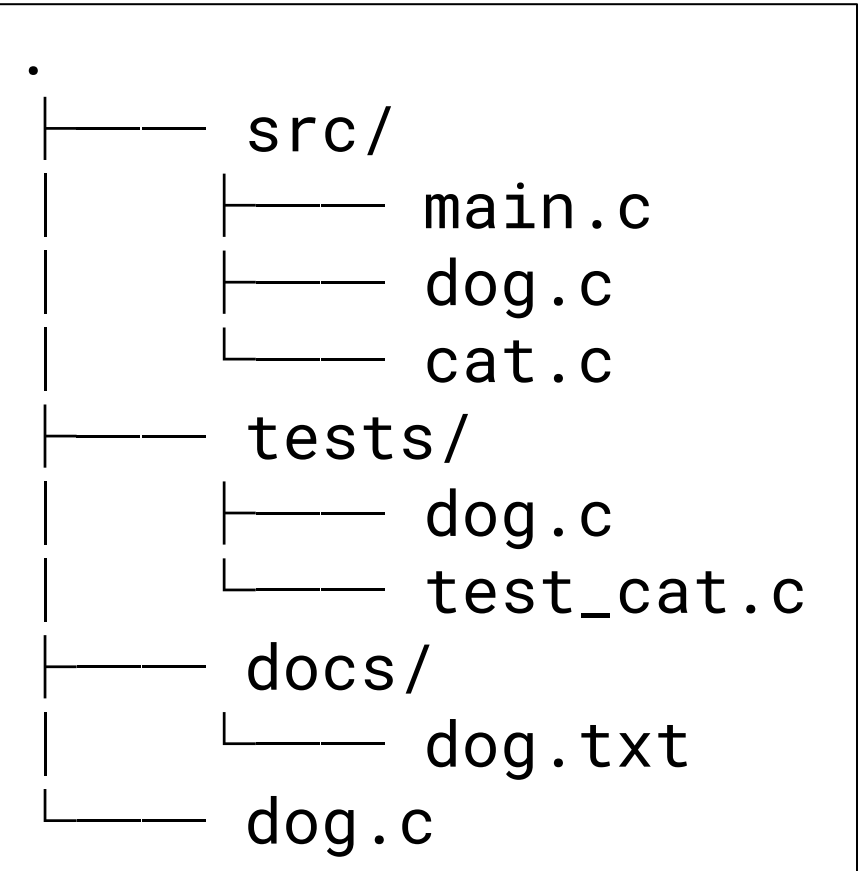
```
.
├── src/
│   ├── main.c
│   ├── dog.c
│   └── cat.c
├── tests/
│   ├── dog.c
│   └── test_cat.c
├── docs/
│   └── dog.txt
└── dog.c
```

find Example

```
find ./src -name dog.c
```

- ./src start searching inside src/ only
 - -name match files by name
 - dog.c match exactly files named dog.c
- Output

```
./src/dog.c
```



Recall:

Pattern	Meaning
*	any number of characters
?	exactly one character
[abc]	one character from the set
[a-z]	one character in a range

What is the output of the following commands?

- Assume commands are run from /home

```
find ./carla -name "*.c"
```

```
find . -name "comp*"
```

```
find ./john -name "*.c"
```

```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
|   |-- comp211
|       |-- labs
|           |-- lab0.c
|           |-- notes
|               |-- lec1.note
|               |-- lec2.note
|-- comp301
```

What is the output of the following commands?

- Assume commands are run from /home

```
find ./carla -name "*.c"
```

- ./carla/comp211/labs/lab0.c

```
find . -name "comp*"
```

- ./carla/comp211
- ./carla/comp301

```
find ./john -name "*.c"
```

- (no output)

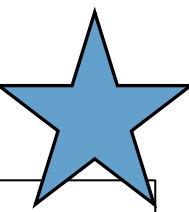
```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
|   |-- comp211
|       |-- labs
|           |-- lab0.c
|           |-- notes
|               |-- lec1.note
|               |-- lec2.note
|-- comp301
```

Find Examples

- Write a `find` command to locate all files ending in `.note` under `/home/john`. Assume command is run from `/`. What would be returned?
- What does the command output? Assume command is run from `/home`
- `find ./carla -name "lab?.c"`

```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
|   |-- comp211
|       |-- labs
|           |-- lab0.c
|           |-- lab1.c
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|-- comp301
```

Examples



- Write a `find` command to locate all files ending in `.note` under `/home/john`. Assume command is run from `/`. What would be returned?
- `find ./home/john -name "*.note"`
 - `./home/john/psyc101/notes/lec1.note`
 - `./home/john/psyc101/notes/lec2.note`
 - `./home/john/psyc101/notes/lec3.note`
- What does the command output? Assume command is run from `/home`
- `find ./carla -name "lab?.c"`
 - `./carla/comp211/labs/lab0.c`
 - `./carla/comp211/labs/lab1.c`

```
/home
|-- john
|   |-- psyc101
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|           |-- lec3.note
|-- carla
|   |-- comp211
|       |-- labs
|           |-- lab0.c
|           |-- lab1.c
|       |-- notes
|           |-- lec1.note
|           |-- lec2.note
|-- comp301
```

The grep Command

- **grep** searches for text patterns inside files
- It scans file contents line by line
- Often used to find:
 - Variable names
 - Error messages
 - Specific words or phrases

grep Syntax

```
grep <pattern> <file>
```

- Example:

```
grep "main" dog.c
```

- Searches for the word `main`
 - Looks inside the file `dog.c`
 - Prints matching lines
- grep uses **regular expression** patterns

grep Example

- Assume we have a file called `log.txt` with the following contents:

```
INFO Server start  
DEBUG Loading configuration  
ERROR Failed to connect to database  
INFO Retrying connection  
ERROR Timeout while connecting  
INFO Server stopped
```

```
grep "ERROR" log.txt
```

- `grep` search text
 - `"ERROR"` regex pattern (simple literal string)
 - `log.txt` file to search
- Output

```
ERROR Failed to connect to database  
ERROR Timeout while connecting
```

grep Example

- The file `users.txt` contains

```
alice_01  
bob_2  
carol_003  
dave_12  
eve_x  
frank_007
```

- Find only the usernames that end with exactly 2 digits.

```
grep "_[0-9]{2}$" users.txt
```

grep Example

- Write a grep command that searches inside `project/data/users.txt` and prints only the lines that:
 - start with a lowercase letter,
 - Followed by any sequence of characters
 - Followed by one underscore (`_`),
 - and end with 2 or 3 digits.
- Assume we are in the parent directory of `project`

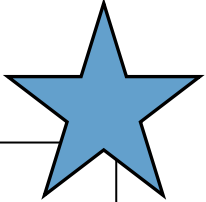
`grep -E`

```
project/
├── src/
│   ├── main.c
│   ├── server.c
│   ├── client.c
│   └── server_test.txt
├── logs/
│   ├── server.log
│   ├── client.log
│   └── old_server.log
├── data/
│   ├── users.txt
│   └── backup_users.txt
├── scripts/
│   ├── deploy.sh
│   └── cleanup.sh
└── README.md
```

grep Example

- Write a grep command that searches inside `project/data/users.txt` and prints only the lines that:
 - start with a lowercase letter,
 - Followed by any sequence of characters
 - Followed by one underscore (`_`),
 - and end with 2 or 3 digits.

```
grep -E "^[a-z].*_[0-9]{2,3}$" project/data/users.txt
```



```
project/  
├── src/  
│   ├── main.c  
│   ├── server.c  
│   └── client.c  
├── server_test.txt  
│   ├── logs/  
│   │   ├── server.log  
│   │   ├── client.log  
│   │   └── old_server.log  
│   ├── data/  
│   │   └── users.txt  
├── backup_users.txt  
│   ├── scripts/  
│   │   ├── deploy.sh  
│   │   └── cleanup.sh  
└── README.md
```

Explanation



- `grep -E "^[a-z].*_[0-9]{2,3}$" project/data/users.txt`

Part	Meaning
<code>^</code>	start of line
<code>[a-z]</code>	starts with lowercase
<code>.</code>	anything (including other underscores)
<code>*</code>	at least one underscore somewhere
<code>_[0-9]{2,3}</code>	ends with 2 or 3 digits
<code>\$</code>	end of line